

FEDERAL INSTITUTE OF ESPÍRITO SANTO
GRADUATE PROGRAM IN APPLIED COMPUTING – PPCOMP

92,0

UÁLACI DOS ANJOS JÚNIOR

**EMPIRICAL EVALUATION OF ECLIPSE SDV AND ROS 2 MIDDLEWARE
PERFORMANCE IN ZONAL ARCHITECTURES UNDER EMULATED
NETWORK IMPAIRMENTS**

UÁLACI DOS ANJOS JÚNIOR

**EMPIRICAL EVALUATION OF ECLIPSE SDV AND ROS 2 MIDDLEWARE
PERFORMANCE IN ZONAL ARCHITECTURES UNDER EMULATED
NETWORK IMPAIRMENTS**

7,0

Preliminary research proposal submitted to the Graduate Program in Applied Computing (PPCOMP) at the Federal Institute of Espírito Santo, as a requirement for passing the Applied Computing Research course.

Advisor: Prof. Dr. Maxwell Eduardo Monteiro

resumo??

Serra
2026

LIST OF FIGURES

Figure 1 – SDV Generations.	5
-------------------------------------	---

LIST OF TABLES

Table 1 – Mapping of Specific Objectives to Measurable Outcomes and Validation Methods	8
Table 2 – Network Degradation Emulation Profiles	16
Table 3 – Chronological Progression and Milestones	19

TABLE OF CONTENTS

1	INTRODUCTION AND PROBLEM FORMULATION	4
1.1	Research Objectives and Methodological Proposal	7
1.1.1	General Objective	7
1.1.2	Specific Objectives	7
2	COMPREHENSIVE THEORETICAL FRAMEWORK	9
2.1	Softwarization, Servitization, and the SDIOV Paradigm	9
2.2	Zonal Architectures and Automotive Ethernet Backbones	10
2.3	Performance Bottlenecks in Incumbent ROS 2 and DDS Middleware	10
2.4	True Zero-Copy IPC Paradigms and iceoryx Limitations	11
2.5	The Eclipse SDV Ecosystem: Zenoh and Kuksa	12
2.6	Stochastic Network Degradation and Teleoperation Modeling	13
3	METHODOLOGICAL DESIGN AND EXPERIMENTAL AR- CHITECTURE	15
3.1	Hardware Architecture and Edge-to-Cloud Continuum	15
3.2	Middleware Provisioning and Workload Emulation	15
3.3	Stochastic Network Emulation Pipeline (tc-netem)	16
3.4	High-Resolution Observability and Telemetry Integration	17
3.5	Statistical Validation Framework	17
4	PROJECT TIMELINE AND EXECUTION MILESTONES . .	19
	REFERENCES	20

formatação errada.
Sugiro seguir o template
disponibilizado.

1 INTRODUCTION AND PROBLEM FORMULATION

The global automotive industry is currently navigating the most profound paradigm shift since the introduction of electronic fuel injection: the transition toward the Software-Defined Vehicle (SDV). This transformation represents a fundamental departure from traditional, hardware-centric electromechanical engineering, moving toward a dynamic, software-centric operational model where vehicle capabilities, safety mechanisms, and user experiences are dictated by extensible, updatable code (1). In this emerging paradigm, vehicle capabilities are no longer static at the point of sale; rather, they evolve continuously throughout the vehicle's lifecycle via Over-The-Air (OTA) updates, enabling novel business models such as Feature-as-a-Service (FaaS) (2). As vehicular connectivity evolves, the Software-Defined Internet of Vehicles (SDIoV) paradigm integrates Software-Defined Networking (SDN) principles into vehicular communication infrastructures, enabling centralized network management, dynamic resource allocation, and improved Quality of Service (QoS) across distributed infrastructure domains (3).

To enable this software-centric vision, the underlying Electrical/Electronic (E/E) topology of modern vehicles is being radically reconstructed (4). As illustrated in Figure 1 (2), SDV Generations, vehicle E/E architectures have evolved through distinct phases in response to increasing computational demands and system complexity. Historically, vehicles relied on a traditional, highly distributed architecture comprising isolated Electronic Control Units (ECUs), where each function was managed independently. In the most advanced cars, there may be over 100 ECUs to cover all functionality (2). This legacy approach resulted in immense point-to-point wiring complexity, introducing severe manufacturing challenges, escalating production costs, and making it exceedingly difficult to modify or upgrade the vehicle after purchase. To address the limitations of fully distributed systems, OEMs initially introduced domain-based architectures, which grouped ECUs into functional domains (e.g., powertrain, body control, infotainment, and ADAS) managed by centralized domain controllers. While this reduced ECU proliferation, communication between domains still required complex network management, and scalability limitations remained when adding new functionalities.

In response to these critical hardware constraints, automotive Original Equipment Manufacturers (OEMs) are aggressively adopting Zonal Architectures. As showed in Figure 1 (2), in a zonal framework, instead of grouping components by functional logic, the vehicle is divided into physical geographic zones (e.g., front-left, rear-right), each managed by a highly capable zone controller. These zone controllers aggregate raw data from localized sensors and actuators, and provide controlled power supply to the components in their designated sub-area. This data is then transmitted over a high-speed, multi-gigabit Automotive Ethernet backbone to a centralized, high-performance computing unit (HPCU)

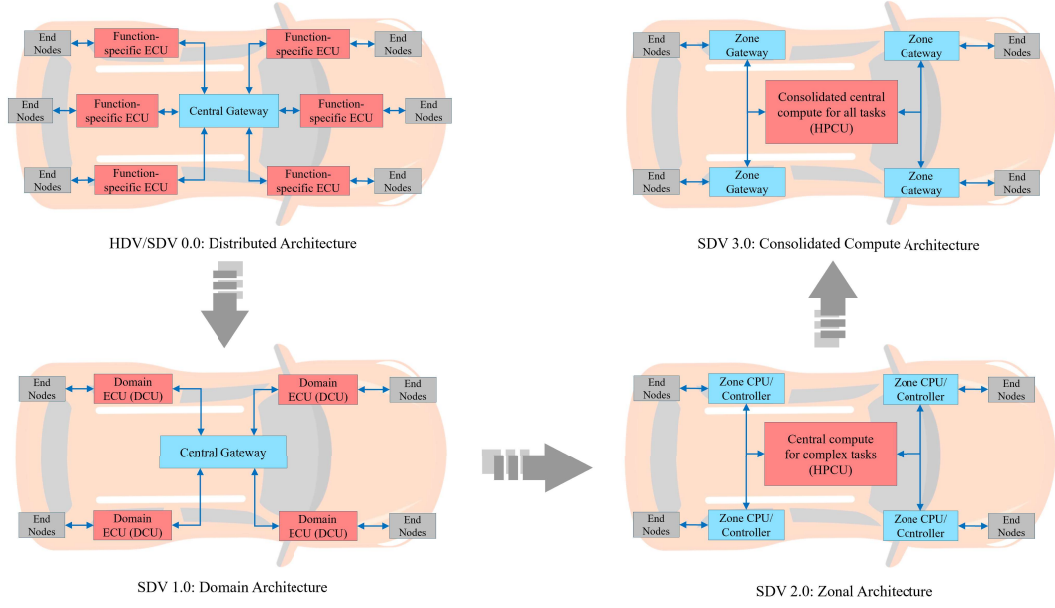


Figure 1 – SDV Generations (adapted from (2)).

located at the heart of the vehicle. The integration of standards such as MultiGBASE-T1 (IEEE 802.3ch) and 10BASE-T1S allows for asymmetric multi-gigabit throughput over a single twisted pair of wires. This structural redesign ensures intelligent task distribution, drastically reduces cabling weight and cost, simplifies communication networks, and enables the seamless integration of software-driven functionalities.

While zonal architectures effectively solve the physical wiring bottleneck and enable the consolidation of computational resources, they inadvertently transfer the immense complexity of mechatronic orchestration directly onto the software communication middleware. Modern software-defined vehicles operate as high-velocity distributed data centers. They generate massive volumes of telemetry and perception data driven by high-bandwidth sensors, (3) such as 4K resolution cameras, LiDARs, and radar arrays. Processing this continuous influx of information locally, while simultaneously synchronizing it with cloud-based Digital Twins for remote teleoperation or predictive fleet management, requires substantial computational power (5). This creates a severe processing and networking bottleneck because these operations must be executed on resource-constrained edge processors, typically ARM-based System-on-Chips (SoCs), which are subjected to strict thermal, power, and financial constraints within the automotive environment (6).

Historically, the Robot Operating System 2 (ROS 2), which is tightly coupled with the Data Distribution Service (DDS) standard, has served as the foundational middleware for academic robotics research and early-stage autonomous vehicle prototyping (6). DDS provides a decentralized, data-centric publish-subscribe architecture equipped with a rich suite of Quality of Service (QoS) policies to govern reliability and latency (6). However, extensive empirical deployments and recent academic literature reveal that standard

ROS 2 and DDS implementations systematically exhaust the CPU cycles and memory bandwidth of ARM-based edge nodes when managing large payloads, such as uncompressed video streams or dense point clouds, in real-time (7). The computational overhead is primarily driven by heavy serialization and deserialization processes, continuous memory copying across process boundaries via the kernel network stack, and the high network cost of maintaining discovery protocols across distributed nodes (7).

This resource exhaustion is heavily increased when the SDV operates over non-deterministic, degraded wide-area communication links, such as cellular 4G/5G networks or Low Earth Orbit (LEO) satellite connections. Under these degraded conditions, incumbent middleware struggles to maintain temporal determinism, resulting in severe latency spikes, packet loss, and ultimate state divergence between the physical vehicle and its cloud-based Digital Twin (8).

To address these critical inefficiencies, a new, highly optimized technological ecosystem is emerging, championed by the open-source Eclipse SDV working group (9). This emergent stack promises to fundamentally redesign automotive middleware for maximum efficiency on constrained edge devices. It is built upon core components such as Eclipse Zenoh (a highly efficient, low-overhead routing and publish-subscribe protocol), Eclipse iceoryx (a true zero-copy inter-process communication framework), and Eclipse Kuksa (a vehicle signal data broker that standardizes telemetry using the COVESA Vehicle Signal Specification) (10).

Despite the rapid industrial momentum behind the Eclipse SDV stack, to the best of our knowledge, there is a scarcity of independent, integrated, end-to-end empirical benchmarks comparing this new ecosystem against the established ROS 2/DDS standard, specifically under conditions of extreme computational stress and deterministic stochastic network degradation. Establishing clear failure thresholds, defining resilience boundaries, and producing validated performance metrics for these competing middleware stacks will allow automotive manufacturers to optimize their hardware pipelines. It prevents the immense financial waste associated with hardware over-provisioning, the practice of selecting excessively powerful, expensive processors simply to compensate for software inefficiencies. Furthermore, mathematically validated temporal determinism is an absolute prerequisite for ensuring the safety of mission-critical operations, such as autonomous driving and remote teleoperation.

Consequently, this research is guided by the following core scientific inquiry: What are the precise quantitative limits of temporal determinism, state consistency, and computational overhead of the emerging Eclipse SDV vehicular middleware pipeline when compared directly to the industry-standard ROS 2 robotics stack, operating on resource-constrained ARM edge architectures under strictly modeled network degradation?

To investigate this, a full-stack integration of Eclipse SDV components (Zenoh, iceoryx, Kuksa) will be deployed on an ARM-based edge node and benchmarked against an equivalent ROS 2/DDS implementation. The experimental design evaluates the simultaneous ingestion of high-density industrial video streams and localized vehicular telemetry, whilst the edge-to-cloud transport layer is actively impaired using Linux kernel traffic control to emulate the exact latency, jitter, and bursty packet loss characteristic of cellular and satellite communications. Sugiro uma seção separada para definição do problema e outra para proposta.

23,0

1.1 Research Objectives and Methodological Proposal

The foundational proposal of this research consists of developing an exhaustive, empirically validated middleware benchmarking framework tailored specifically for the rigorous demands of Software-Defined Vehicles. The core structure is designed as a physical, A/B experimental testbench evaluating edge-to-cloud communication paradigms. By orchestrating continuous, mixed-criticality synthetic workloads and systematically emulating degraded wide-area transport conditions in a highly controlled environment, the project will furnish the academic and industrial communities with a statistically validated comparison of the computational efficiency and fault-resilience frontiers of both the Eclipse SDV stack and the ROS 2/DDS architecture.

1.1.1 General Objective

To evaluate empirically the computational performance, resource utilization efficiency, and state synchronization resilience of the integrated Eclipse SDV pipeline (comprising Zenoh, iceoryx, and Kuksa) in direct comparison to the established ROS 2 (DDS) standard, operating on a resource-constrained embedded edge node under strictly emulated conditions of degraded, non-ideal network connectivity. 10,0

1.1.2 Specific Objectives

To ensure a highly structured, measurable, and reproducible methodological approach, the general objective is decomposed into specific, actionable objectives. Table 1 maps each specific objective to its corresponding measurable outcome and the scientific validation method that will be employed to ensure empirical rigor.

Tabela 1 – Mapping of Specific Objectives to Measurable Outcomes and Validation Methods

Specific Objective	Measurable Outcome & Artifact	Validation Method
To implement a physical edge-to-cloud testbench integrating a resource-constrained ARM-based edge node with an x86_64 cloud server architecture.	A fully operational, bidirectionally networked hardware cluster capable of deploying and executing containerized middleware stacks.	Network topology verification via ICMP profiling and sustained maximum throughput baseline testing via standardized network utilities.
To configure mixed-criticality workload generators capable of emitting simultaneous high-bandwidth data (video payloads) and low-latency vehicular signals.	Automated payload generation scripts simulating realistic autonomous vehicle sensor pipelines at precisely modulated frequencies (e.g., 30Hz and 100Hz).	Workload validation confirming stable generation frequencies and byte-accurate payload sizes prior to network transmission.
To modulate deterministic network degradations on the communication interface using the Linux kernel traffic control emulator (tcnetem) utilizing Gilbert-Elliott stochastic modeling.	Configured QoS classes and queuing disciplines (qdiscs) enforcing precise delay, jitter, bandwidth capping, and bursty packet loss simulating 4G, 5G, and satellite networks.	Packet capture analysis (Wireshark/tcpdump) confirming that the applied jitter and loss distributions match the theoretical mathematical models.
To collect quantitative performance metrics, specifically end-to-end latency, jitter, CPU consumption percentiles, and memory footprint, across both competing middleware platforms.	A highly granular, time-series dataset captured via kernel-space and user-space tracing (LTTng and ros2_tracing), exported via Prometheus.	Data integrity checks ensuring zero dropped tracepoints and accurate clock synchronization between edge and cloud nodes using Precision Time Protocol (PTP).
To investigate the micro-architectural and protocol-level causes justifying the observed resource consumption disparities between the two middleware ecosystems.	Statistical variance models isolating the root causes of latency spikes, thread contention, and CPU saturation under specific network degradation states.	Two-way Analysis of Variance (ANOVA) or Kruskal-Wallis hypothesis testing, complemented by rigorous post-hoc statistical analysis.
To formulate a comprehensive recommendation matrix based on the empirical trade-offs of each stack, indicating optimal deployment scenarios for automotive engineering.	A decision-support framework detailing middleware suitability as a mathematical function of payload size, compute budget, and network quality.	Peer-review validation through academic dissemination of the proposed multi-dimensional matrix.

2 COMPREHENSIVE THEORETICAL FRAMEWORK

To properly contextualize the proposed empirical benchmarking, this section establishes the theoretical foundation intersecting the architectural evolution of automotive manufacturing with the stringent constraints of distributed computing, middleware protocol mechanics, and stochastic network degradation modeling. The convergence of these domains fundamentally defines the operational landscape of the modern Software-Defined Vehicle (SDV).

2.1 Softwarization, Servitization, and the SDIOV Paradigm

The fundamental premise of the SDV is extensively analyzed in recent literature dissecting the automotive industry's transition from a traditional model of hardware ownership to a paradigm of software-driven, service-centric platforms.⁽²⁾ This process of "softwarization" yields immense strategic advantages, primarily the enablement of Feature-as-a-Service (FaaS) business models. In this paradigm, a vehicle's core capabilities, ranging from advanced driver-assistance systems (ADAS) to battery management optimizations and infotainment upgrades, can be deployed dynamically via Over-The-Air (OTA) updates throughout its lifecycle.⁽²⁾ This circumvents the historical requirement for physical hardware alterations to upgrade vehicle functionality. Furthermore, softwarization facilitates deep integration between the automotive industry and cloud computing ecosystems, drastically accelerating the development of advanced safety systems through cloud-based simulations, shadow mode deployments, and the utilization of Digital Twins.⁽⁵⁾

This evolution has given rise to the Software-Defined Internet of Vehicles (SDIoV). As vehicular connectivity evolves, the SDIoV paradigm integrates Software-Defined Networking (SDN) principles into vehicular communication infrastructures. This enables centralized network management, dynamic resource allocation, and improved Quality of Service (QoS) across distributed infrastructure domains.⁽³⁾ Architecturally, SDVs function as cyber-physical nodes within Intelligent Transportation Systems (ITS), supporting Vehicle-to-Everything (V2X) communication, cooperative driving, real-time traffic optimization, and continuous OTA-driven software evolution.⁽³⁾ However, the SDV integration into the broader IoT ecosystem has exponentially increased data volumes and introduced profound engineering challenges related to processing bottlenecks, decentralized storage, and deterministic system reliability.⁽³⁾ Modern software-defined vehicles now execute massive codebases, necessitating a fundamental departure from legacy automotive network topologies.

2.2 Zonal Architectures and Automotive Ethernet Backbones

As automotive manufacturers abandon traditional, decentralized domain-based architectures, the reliance on a central, high-speed networking backbone becomes absolute.(4) In a zonal architecture, the vehicle’s hardware is abstracted geographically rather than functionally. Raw sensor data, previously processed locally by a dedicated, function-specific ECU, is now aggregated by a regional controller and forwarded via an Automotive Ethernet backbone to a central High-Performance Computing (HPC) cluster.(11)

This architectural consolidation relies heavily on the advancement of in-vehicle networking standards. Legacy Controller Area Network (CAN) and Local Interconnect Network (LIN) protocols are insufficient for the bandwidth required by SDVs. Consequently, technologies such as Single-Pair Ethernet (SPE), 10BASE-T1S, and MultiGBASE-T1 (IEEE 802.3ch) have emerged as the foundational layers for zonal communication.(12) These standards provide asymmetric multi-gigabit throughput over a single twisted pair of wires, significantly reducing harness weight while ensuring strict electromagnetic compatibility and supporting Power over Data Line (PoDL) capabilities.(12) Furthermore, modern Automotive Ethernet leverages Time-Sensitive Networking (TSN) to guarantee bounded latency, supporting service-oriented communication frameworks that are highly critical for Advanced Driver-Assistance Systems (ADAS) and autonomous navigation.(13) This consolidation places an unprecedented burden on the communication middleware, which must now route massive, heterogeneous payloads deterministically while strictly adhering to safety-critical TSN constraints.(13)

2.3 Performance Bottlenecks in Incumbent ROS 2 and DDS Middleware

The Robot Operating System 2 (ROS 2), backed by the Data Distribution Service (DDS) standard, has historically served as the foundational software architecture for complex robotic systems and autonomous vehicle research.(6) DDS operates on a decentralized publish/subscribe model, utilizing the Real-Time Publish-Subscribe (RTPS) wire protocol to enable dynamic node discovery and data exchange without a central message broker.(6) While highly effective and resilient in stable, wired local area networks, a growing body of recent literature highlights critical bottlenecks when DDS is deployed on constrained edge nodes handling the heavy workloads typical of SDVs.(7)

Empirical studies demonstrate these severe limitations in autonomous vehicle perception pipelines running on embedded architectures. When multiple high-frequency sensor nodes and control algorithms share a single computing unit, ROS 2 and its underlying DDS transports introduce profound serialization and deserialization overheads.(7) Standard ROS 2 middleware architecture defaults to copying data across process boundaries via the kernel network stack, which severely erodes the available real-time compute budget

by forcing the CPU to continuously allocate memory and execute context switches.(7) Consequently, research demonstrates that the internal latency layers of ROS 2 shared memory communication introduce measurable delays; the overhead introduced by ROS 2 compared to raw, low-level DDS communication can reach up to 50% for small messages, driven largely by executor scheduling mechanisms and thread contention.(14)

These inefficiencies become critical failures under degraded wide-area network conditions. DDS suffers severe throughput collapse and latency spikes over Wi-Fi and 4G networks due to the "chattiness" of the DDS Simple Participant Discovery Protocol (SPDP) and its strict state-matching mechanisms, which aggressively consume bandwidth and CPU cycles when network packets are dropped or delayed.

2.4 True Zero-Copy IPC Paradigms and iceoryx Limitations

To bypass the serialization bottlenecks inherent in legacy DDS, the industry has shifted toward true zero-copy Inter-Process Communication (IPC) paradigms.(7) For intra-host communication, Eclipse iceoryx provides a lock-free, zero-copy shared memory framework originally developed for safety-critical real-time systems in autonomous driving.(15) Unlike standard DDS, which serializes a multi-megabyte video frame, pushes it to the network stack, and deserializes it for the subscriber, iceoryx allows the publisher to write data directly into a pre-allocated shared memory segment.(7) Subscribers receive only a lightweight smart pointer to the data location, ensuring that transmission time remains strictly constant regardless of the payload size.(16)

However, current zero-copy implementations like iceoryx possess strict architectural constraints. They require memory pools to be pre-allocated before runtime, utilizing static-sized message formats.(7) Due to the zero-copy approach, transmitted data types must be fully representable in shared memory; dynamically growing types that rely on heap memory, such as strings or vectors from the C++ standard library, cannot be used directly.(7) When forced to communicate unsized message types, such as the PointCloud2 messages pervasive in the ROS 2 Autoware autonomous driving stack, iceoryx cannot perform true zero-copy.(7) It incurs transparent serialization and deserialization costs to and from the shared memory. As a result, its latency increases exponentially as the message size grows, occasionally yielding latency higher than standard CycloneDDS-based communication.(7)

Recent advancements, such as the Agnocast framework, have attempted to solve this by providing true zero-copy IPC for unsized message types in ROS 2 C++ on Linux without requiring developer-defined static arrays.(7) When applied to the PointCloud Preprocessing component of Autoware, Agnocast demonstrated a 16% improvement in average response time and a 25% improvement in worst-case response time compared to standard architectures.(7) Furthermore, under intense CPU load testing utilizing stress-ng

(varying CPU utilization from 0% to 90% while utilizing `SCHED_FIFO` thread scheduling policies), true zero-copy solutions maintained constant communication latency regardless of payload size.(7) Evaluating how the Eclipse SDV stack manages these precise memory constraints under high-frequency sensor loads is a critical component of assessing its viability for automotive engineering.

2.5 The Eclipse SDV Ecosystem: Zenoh and Kuksa

While zero-copy frameworks handle local inter-process communication, edge-to-cloud routing and semantic orchestration require entirely different middleware solutions. The Eclipse SDV working group addresses this through Eclipse Zenoh and Eclipse Kuksa.(9)

Eclipse Zenoh is a novel publish/subscribe and distributed query protocol designed to seamlessly unify data-in-motion, data-at-rest, and computational routing. Written in Rust, it is explicitly optimized to minimize network protocol overhead and support extremely constrained microcontrollers and edge nodes. Benchmarks comparing Zenoh against DDS, MQTT, and Kafka indicate that Zenoh dramatically outperforms the incumbent technologies in both throughput and latency.(10) Under comparative testing using both brokered and peer-to-peer topologies, Zenoh demonstrated latency as low as 13 μ s, heavily outperforming MQTT and standard DDS configurations in multi-machine environments.(10) Zenoh’s advanced routing algorithms mitigate the discovery storms associated with DDS SPDP, providing significantly greater resilience to packet loss and jitter over unstable wireless networks.

Operating cohesively with Zenoh is Eclipse Kuksa, which provides the crucial semantic data orchestration layer.(9) Kuksa utilizes the COVESA Vehicle Signal Specification (VSS) to abstract low-level vehicle hardware signals (e.g., CAN bus data, steering angles, brake pressure) into a standardized, hierarchical data model.(17) The VSS acts as a common, protocol-agnostic language describing vehicle data both in the physical vehicle and in the cloud.(18)

The Kuksa Databroker, implemented as a highly concurrent gRPC service in Rust, allows vehicle applications to subscribe to specific telemetry data points efficiently.(18) Data providers (historically referred to as feeders) sync the actual state of the vehicle to the VSS model, while actuation providers sync the desired state from the VSS server back to the vehicle’s lower-level systems.(19) This architecture standardizes edge-to-cloud telemetry, enabling seamless integration with continuous delivery pipelines, fleet management architectures, and digital twin creation in environments like Unreal Engine 5.(5) Furthermore, utilizing microservice patterns (such as the Kuksa* self-adaptive framework), the architecture employs load balancing and resiliency mechanisms to improve self-adaptive quality performance under the dynamic conditions of the automotive domain.(20)

2.6 Stochastic Network Degradation and Teleoperation Modeling

In the context of modern SDVs, transmitting telemetry actively synchronizes Digital Twins in the cloud for predictive modeling, route planning, and remote teleoperation. (5) The temporal consistency of these operations is highly vulnerable to the latency and throughput fluctuations inherent in wireless mobility networks (e.g., 5G cell handovers, LTE edge-of-cell degradation). As demonstrated by the VISTA benchmark for real-time video streaming in teleoperation, network impairments directly impact task completion and system safety, heavily prioritizing predictable latency over pure bandwidth. (8) Research utilizing the NASA-TLX framework indicates that a 320 ms latency threshold is the maximum acceptable limit for safe teleoperation; beyond this, task performance degrades exponentially, even if video quality metrics (such as PSNR, SSIM, and VMAF) remain high. (21)

When wide-area network transport is degraded, out-of-order packets and severe jitter destroy the temporal determinism required for state synchronization. To accurately benchmark middleware under these conditions, researchers must utilize advanced network emulation. The Linux kernel’s Traffic Control Network Emulator (tc-netem) provides the capability to strictly emulate wide-area networks by applying delay, jitter, and bursty packet loss based on highly accurate stochastic models applied directly to the network interface card. (22)

Unlike simple random packet loss generation, realistic wireless environments exhibit bursty, autocorrelated loss profiles resulting from signal blockage or interference. (23) This behavior is mathematically modeled using the Gilbert-Elliott (GE) stochastic model, a two-state Markov chain implemented natively in tc-netem. (22)

Conceptually, the GE model simulates a network alternating between a mostly reliable state and a highly degraded state. The severity of the network failure depends on how long the system remains trapped in the degraded state and the specific packet drop rates assigned to each condition. The model consists of a Good (low-loss) state and a Bad (high-loss) state, governed by four primary parameters (22):

- p (**Transition to Bad**): The probability of the network degrading from the Good state into the Bad state.
- r (**Transition to Good**): The probability of the network recovering from the Bad state back into the Good state.
- $1 - k$ (**Good State Loss**): The probability of an isolated packet drop while the network is in the Good state (where k represents successful transmission).

- **$1 - h$ (Bad State Loss):** The probability of a packet dropping while the network is actively in the Bad state (where h represents successful transmission).

By utilizing these parameters, the model accurately recreates both isolated loss events and clustered, bursty loss events. To evaluate a generated sequence of n (the total number of transmitted packets), we calculate the mathematical mean (μ), which represents the expected number of total lost packets:

$$\mu = n \frac{r(1 - k) + p(1 - h)}{p + r} \quad (1)$$

Furthermore, the variance (σ^2) defines the spread or "burstiness" of that loss distribution. A higher variance indicates that losses are clustering together rather than spacing out evenly:

$$\sigma^2 = n \left[\frac{r(1 - k) + p(1 - h)}{p + r} \right]^2 \left\{ \frac{hp + kr}{(1 - h)p + (1 - k)r} + \frac{2pr(1 - p - r)(h - k)^2}{[(1 - h)p + (1 - k)r]^2(p + r)} \left[1 - \frac{1 - (1 - p - r)^n}{n(p + r)} \right] \right\} \quad (2)$$

In the kernel-space implementation (`sch_netem.c`), these parameters are passed via netlink sockets under `NETEM_LOSS_GE`. The state machine transition maps `clg->a1` to p , `clg->a2` to r , `clg->a3` to h , and `clg->a4` to k . The logic determines packet drops dynamically based on the active state transition. (22) By enforcing these mathematical models within the testbench, the research guarantees a highly deterministic evaluation of how middleware stacks handle realistic, correlated transport failure without triggering discovery protocol broadcast storms.

Explicitar a lacuna da literatura

3 METHODOLOGICAL DESIGN AND EXPERIMENTAL ARCHITECTURE

To rigorously evaluate the computational efficiency, scalability, and resilience of the competing middleware platforms, this research relies on an applied, quantitative, and experimental methodology. The technical procedures revolve around a highly controlled physical testbench utilizing industry-standard network emulation protocols and nanosecond-precision observability frameworks. The methodological approach is strictly quantitative, relying entirely on the statistical analysis of machine-generated time-series data to draw objective conclusions regarding software efficiency.

3.1 Hardware Architecture and Edge-to-Cloud Continuum

The physical testbench is architected as an edge-to-cloud continuum, comprising two primary computational nodes designed to accurately reflect a realistic SDV telemetry pipeline: The Edge Node acts as a Zonal Gateway Proxy. To accurately represent the thermal and computational constraints of modern zonal controllers, the experiment utilizes an ARM-based embedded processor (e.g., an NVIDIA Jetson architecture featuring ARM Cortex CPUs). This architecture is selected due to its ubiquitous presence in both academic robotics and industrial autonomous driving prototyping. The Cloud Node acts as the Digital Twin Proxy. An x86_64 architecture server with substantial compute resources (e.g., Intel Core i9 or AMD EPYC, minimum 24GB RAM) serves as the receiving telemetry endpoint and the Digital Twin state manager. The two nodes are physically connected via a dedicated Gigabit Ethernet interface. It is exclusively over this localized link that the wide-area network emulation constraints are artificially applied, ensuring total isolation from external network noise and maximizing experimental reproducibility.

3.2 Middleware Provisioning and Workload Emulation

The experiment is structured as a comparative A/B test, evaluating two distinct middleware pipelines orchestrating the exact same underlying workloads. In the Control Group, the ARM edge node executes a modern Long Term Support (LTS) distribution of ROS 2. The underlying middleware utilizes a default, widely adopted DDS implementation (such as Cyclone DDS or FastDDS). The stack is configured for both local inter-process communication, utilizing standard ROS 2 shared memory plugins, and wide-area network telemetry publishing.⁽⁶⁾ In the Experimental Group, the edge node executes the highly integrated Eclipse SDV stack. Intra-host communication for heavy, large-byte payloads is managed by Eclipse iceoryx to enforce lock-free, zero-copy memory exchange, strictly bypassing the kernel network stack.⁽⁷⁾ Localized vehicle telemetry is ingested via the Eclipse Kuksa Databroker, mapping disparate signals to the unified COVESA VSS standard using gRPC interfaces.⁽¹⁸⁾ Finally, edge-to-cloud routing and state synchronization are handled exclusively by the Eclipse Zenoh router, which bridges the Kuksa data

and routes it over the wide-area network.(10) To accurately simulate the mixed-criticality processing demands of an autonomous SDV, the testbench utilizes automated workload generators to emit two continuous, concurrent data streams. The first is a High-Bandwidth Perception Stream involving the continuous generation of large payloads (ranging from 2MB to 5MB byte arrays) published at high frequencies (30Hz to 60Hz). This mimics the uncompressed video frames or dense LiDAR point clouds generated by autonomous perception sensors. The second is a Low-Latency Telemetry Stream consisting of high-frequency, small-payload data (e.g., 100 to 250 bytes) published at 100Hz. This mimics internal CAN bus signals (e.g., wheel speed, steering angle, braking pressure) mapped into the VSS JSON format via Kuksa.(17) This dual-stream approach forces the middleware to multiplex massive throughput demands alongside latency-sensitive state updates, pushing the edge CPU to its architectural limits.

3.3 Stochastic Network Emulation Pipeline (tc-netem)

To evaluate network resilience, the Gigabit Ethernet link between the Edge and Cloud nodes is systematically degraded using the Linux kernel’s Traffic Control (tc) system, specifically leveraging the netem queuing discipline. This tool injects precise impairments directly into the kernel network stack, avoiding the need for expensive hardware emulators.(22)

The emulation is structured across specific QoS tiers modeled on empirical telecommunication data derived from real-world network testing. A Hierarchical Token Bucket (HTB) first enforces strict bandwidth limits. Subsequently, netem applies latency, jitter, and packet loss. Jitter is applied using a normal distribution to reflect realistic queue variations, while packet loss for wireless networks is strictly modeled using the Gilbert-Elliott Markov chain to simulate cellular handovers and signal interference accurately.(22)

Table 2 details the target emulation profiles applied during the continuous test phases.

Tabela 2 – Network Degradation Emulation Profiles

Network Profile	Emulated Technology	Bandwidth Limit	Base Latency (RTT)	Jitter ($\pm$)	Packet Loss Model (Gilbert-Elliott)
Baseline	Stable Gigabit LAN	1000 Mbps	1 ms	0.1 ms	0.00% (Lossless)
Profile A	5G Urban	50 Mbps	20 ms	5 ms	0.1% (Low burst probability)
Profile B	4G Rural / Cell Edge	5 Mbps	120 ms	40 ms	1.5% to 5.0% (High burst probability)
Profile C	LEO Satellite	30 Mbps	45 ms	15 ms	0.5% (Moderate burst probability)

3.4 High-Resolution Observability and Telemetry Integration

Accurate evaluation of highly concurrent middleware requires nanosecond-precision tracing with near-zero overhead. Standard monitoring tools frequently artificially inflate latency, a phenomenon known as the observer effect, rendering real-time performance data invalid. To mitigate this, the primary observability framework utilizes the Linux Trace Toolkit: next generation (LTTng). For the ROS 2 group, the `ros2_tracing` framework instruments the middleware layers directly, providing thread-safe, lock-free tracepoints that bypass system calls in the fast path.(24) The architecture divides instrumentation into two phases: Initialization Events, which collect one-time information about node creation, and Runtime Events, which collect active execution data using unique identifiers mapped back to initialization.(24) This reduces the payload size of active tracepoints. The `tracetools` package serves as an indirection layer, keeping the core code free of boilerplate macros.(24) Empirical evaluation of this tracing architecture demonstrates that the runtime cost of a single LTTng userspace tracepoint is approximately 158 nanoseconds.(24) With roughly 10 tracepoints placed in the standard message publish-subscribe hot path, the theoretical overhead is calculated at merely 1,580 nanoseconds. Empirical end-to-end latency overhead under full instrumentation is validated at approximately 0.0033 ms on average, ensuring that the observability framework is highly suitable for profiling mission-critical, real-time systems without skewing the underlying metrics.(24) Equivalent LTTng tracing is applied to the Eclipse SDV stack. Resource utilization metrics, including CPU consumption (user and system space), memory footprint (Resident Set Size), and context switch rates, are collected using the Linux `perf` utility. This data is ingested and exposed via Prometheus, utilizing Grafana for the real-time visualization and aggregation of the time-series data. The primary dependent variables extracted from the telemetry infrastructure include End-to-End Latency, Jitter Variance (specifically measuring 95th and 99th percentile tail latencies), CPU/RAM Utilization, and State Convergence Time (the delay before the cloud Digital Twin achieves total parity with the edge node's state following a severe network disruption).

3.5 Statistical Validation Framework

The experimental design generates a massive, multi-dimensional time-series dataset. To ensure scientific rigor and reject the null hypothesis, a highly structured statistical validation pipeline is applied. Data distributions for latency and CPU usage are initially tested for normality using the Shapiro-Wilk test, and for homoscedasticity (equality of variances) using Levene's test. Given the high likelihood of heavily skewed right-tail latencies, especially under degraded 4G rural network profiles where tail latencies can exhibit long-tail distributions reflecting intermittent connectivity loss, log-transformations may be applied to normalize the data prior to variance testing. The core evaluation utilizes a

two-way Analysis of Variance (ANOVA) if parametric assumptions are successfully met, or the non-parametric Kruskal-Wallis H test if the data remains skewed. The independent variables (factors) are the Middleware Stack (ROS 2/DDS vs. Eclipse SDV) and the Network Profile (Baseline, Profile A, B, C). The dependent variables evaluated are mean latency, 99th percentile tail latency, and CPU usage percentiles. If the ANOVA reveals statistically significant main effects or interaction effects, rigorous post-hoc tests (such as Tukey's Honestly Significant Difference for ANOVA, or Dunn's test for Kruskal-Wallis) are executed. This determines exactly which specific network conditions trigger catastrophic performance divergence between the two stacks. Finally, Pearson or Spearman correlation coefficients are calculated to map the mathematical relationship between CPU saturation spikes and packet loss events, quantifying the exact threshold at which architectural inefficiencies in the middleware, such as DDS SPDP discovery storms or unbounded serialization overhead, cause systemic failure on the ARM edge processor.

4 PROJECT TIMELINE AND EXECUTION MILESTONES

The execution of this comprehensive empirical evaluation is structured over an 18-month academic timeline, systematically divided into literature review, infrastructure development, experimentation, and documentation phases. Table 3 outlines the chronological progression and expected milestones.

10,0

Tabela 3 – Chronological Progression and Milestones

Project Phase	Duration	Core Activities and Milestones
Phase 1: Theoretical Mapping	Months 1 – 3	Conduct exhaustive literature review on SDV middleware, zero-copy IPC, and Digital Twin synchronization. Consolidate the theoretical framework and define exact performance baselines.
Phase 2: Testbench Setup	Months 4 – 5	Procure ARM edge hardware and x86 cloud nodes. Install base operating systems, configure real-time Linux kernels, and setup container runtimes.
Phase 3: Middleware Implementation	Months 6 – 8	Deploy ROS 2/DDS architecture. Deploy Eclipse SDV stack (Zenoh, iceoryx, Kuksa). Develop workload generators for video and VSS payloads.
Phase 4: Network Emulation & Data Collection	Months 9 – 11	Configure tc-netem QoS profiles applying Gilbert-Elliott mathematical models. Execute continuous A/B test runs. Capture granular performance data via <code>ros2_tracing</code> , LTTng, and Prometheus.
Phase 5: Statistical Analysis	Months 12 – 13	Aggregate time-series data. Perform Shapiro-Wilk, ANOVA, and post-hoc statistical validations. Identify exact performance frontiers and failure thresholds.
Phase 6: Manuscript Writing	Months 14 – 16	Synthesize empirical findings into the final academic manuscript. Draft the comprehensive recommendation matrix based on hardware/network trade-offs.
Phase 7: Review and Dissemination	Months 17 – 18	Finalize formatting, ensure open-source dataset publication, process peer-review revisions, and prepare the formal presentation for the defense committee.

This structured progression ensures that the transition from infrastructural setup to rigorous statistical analysis is paced appropriately, leaving ample time for the documentation of complex quantitative findings. The ultimate outcome of this research will provide the automotive engineering community with the first holistic, mathematically validated comparison of Eclipse SDV against incumbent technologies. By explicitly mapping the resilience and performance boundaries of these middleware stacks under realistic, stochastically degraded network conditions, this work will directly inform the architectural design of next-generation, safety-critical software-defined vehicles, minimizing computational overhead and maximizing operational determinism.

REFERENCES

- 1 SOFTWARE defined vehicles: Driving the future of mobility and monetization - KPMG International. 2026. Available at: https://assets.kpmg.com/content/dam/kpmgsites/se/pdf/2026/software_defined_vehicles_coe26.pdf.
- 2 KHAMIS, A.; GOSWAMI, P. Rethinking vehicle architecture through softwarization and servitization. *IEEE Access*, v. 13, p. 126213–126226, 2025.
- 3 THE Rise of the Software-Defined Vehicle: Architectures, Enabling Technologies, and Future Opportunities - arXiv. 2026. Available at: <https://arxiv.org/html/2605.30001v1>.
- 4 A Scalable Domain Zonal Architecture for Next Generation Software Defined Vehicles. 2026. Available at: <https://ijrdst.org/public/uploads/paper/117451777364449.pdf>.
- 5 TIMONEN, O. et al. Kuura: Leveraging eclipse kuksa in vehicular data collection and digital twin creation environment. In: . [S.l.: s.n.], 2024.
- 6 BONCI, A. et al. Ros 2-based architecture for autonomous driving systems: Design and implementation. *Sensors*, v. 26, n. 2, 2026. Available at: <https://www.mdpi.com/1424-8220/26/2/463>.
- 7 ISHIKAWA-ASO, T.; KATO, S. *ROS 2 Agnocast: Supporting Unsized Message Types for True Zero-Copy Publish/Subscribe IPC*. 2025. Available at: <https://arxiv.org/abs/2506.16882>.
- 8 DENG, Z. et al. *VISTA: A Benchmark for Real-Time Video Streaming under Network Impairments in Surgical Teleoperation*. 2026. Available at: <https://arxiv.org/abs/2605.08886>.
- 9 LEARN about Eclipse Kuksa’s journey - Eclipse SDV. 2026. Available at: <https://eclipsesdv.org/blogs/eclipse-kuksa-the-first-step-towards-building-an-sdv-system/>.
- 10 COMPARING the Performance of Zenoh, MQTT, Kafka, and DDS. 2026. Available at: <https://zenoh.io/blog/2023-03-21-zenoh-vs-mqtt-kafka-dds/>.
- 11 ETHERNET-BASED high-speed communication for Zonal Architectures and Telematics systems - Toshiba Electronic Devices & Storage. 2026. Available at: https://toshiba.semicon-storage.com/content/dam/toshiba-ss-v3/master/en/semiconductor/design-development/innovationcentre/whitepapers/TCM0638_%20TC9563XBG_ZonalEEArchitectures.pdf.
- 12 ACCELERATING Automotive Connectivity: The Rise of MultiGBASE-T1 Ethernet | InterOperability Laboratory - UNH-IOL. 2025. Available at: <https://www.iol.unh.edu/blog/2025/07/24/accelerating-automotive-connectivity-rise-multibase>.
- 13 THE Role of Automotive Ethernet in Smart and Connected Electric Vehicles - ResearchGate. 2026. Available at: https://www.researchgate.net/publication/404921042_The_Role_of_Automotive_Ethernet_in_Smart_and_Connected_Electric_Vehicles.
- 14 BEDARD, C.; LUTKEBOHLE, I.; DAGENAIS, M. ros2_tracing: Multipurpose low-overhead framework for real-time tracing of ros 2. *IEEE Robotics and Automation Letters*, v. 7, n. 3, p. 6511–6518, 2022.

- 15 BARTH, T. et al. *OPC UA Shared-Memory: Conceptual Elaboration and Prototypical Implementation Using Iceoryx2*. 2026. Available at: <https://arxiv.org/abs/2606.21328>.
- 16 ECLIPSE iceoryx - true zero-copy inter-process-communication - GitHub. 2026. Available at: <https://github.com/eclipse-iceoryx/iceoryx>.
- 17 VEHICLE Signal Specification - COVESA. 2026. Available at: <https://covesa.global/vehicle-signal-specification/>.
- 18 IN-VEHICLE access to standardized VSS Vehicle Signals. 2026. Available at: https://archive.fosdem.org/2023/schedule/event/kuksa/attachments/slides/5650/export/events/attachments/kuksa/slides/5650/KUKSA_Fosdem2023.pdf.
- 19 KUKSA.VAL/DOC/SYSTEM-ARCHITECTURE.MD at master · eclipse-archived/kuksa.val · GitHub. 2026. Available at: <https://github.com/eclipse/kuksa.val/blob/master/doc/system-architecture.md>.
- 20 BANIJAMALI, A. et al. *Kuksa*: Self-Adaptive Microservices in Automotive Systems*. 2021.
- 21 DENG, Z. et al. *VISTA: A Benchmark for Real-Time Video Streaming under Network Impairments in Surgical Teleoperation*. 2026. Available at: <https://arxiv.org/abs/2605.08886>.
- 22 LUDOVICI, F.; ORDINE, A. Definition of a general and intuitive loss model for packet networks and its implementation in the linux kernel netem. 2012.
- 23 AMARAL, J. do; SOFIA, R. C. *Building a Low-cost Network Digital Twin for the IoT-Edge-Cloud Continuum Using Open-Source Tooling*. 2026. Available at: <https://arxiv.org/abs/2606.24853>.
- 24 BEDARD, C.; LUTKEBOHLE, I.; DAGENAIS, M. ros2_tracing: Multipurpose low-overhead framework for real-time tracing of ros 2. *IEEE Robotics and Automation Letters*, v. 7, n. 3, p. 6511–6518, 2022.

Repetido!!